

Experimental Comparison

การทดลองนี้มีวัตถุประสงค์เพื่อเปรียบเทียบประสิทธิภาพของ Algorithm A: FIFO Queue และ Algorithm B: Priority Queue
เมื่อมีจำนวน Event แตกต่างกัน โดยวัดเวลาการทำงานด้วย System.nanoTime()

ขนาดข้อมูลที่ใช้ในการทดลอง

กำหนดจำนวน Event เป็น

- 100 รายการ
- 1,000 รายการ
- 10,000 รายการ
- 50,000 รายการ

แต่ละขนาดข้อมูลทำการทดลอง 5 รอบ แล้วนำเวลาที่ได้มาคำนวณค่าเฉลี่ย เพื่อให้ผลการทดลองมีความน่าเชื่อถือมากขึ้น

จำนวน Event (n)	FIFO รอบที่ 1 (ns)	FIFO รอบที่ 2 (ns)	FIFO รอบที่ 3 (ns)	FIFO รอบที่ 4 (ns)	FIFO รอบที่ 5 (ns)	FIFO เฉลี่ย (ns)	Priority รอบที่ 1 (ns)	Priority รอบที่ 2 (ns)	Priority รอบที่ 3 (ns)	Priority รอบที่ 4 (ns)	Priority รอบที่ 5 (ns)	Priority เฉลี่ย (ns)
100	-	-	-	-	-	-	-	-	-	-	-	-
1,000	-	-	-	-	-	-	-	-	-	-	-	-
10,000	-	-	-	-	-	-	-	-	-	-	-	-
50,000	-	-	-	-	-	-	-	-	-	-	-	-

ns = Nanoseconds หรือหน่วยนาโนวินาที

วิธีการทดลอง

1. สร้างข้อมูล

สร้าง Event จำนวน 100, 1,000, 10,000 และ 50,000 รายการ โดยใช้ข้อมูลที่สร้างขึ้นอัตโนมัติ

2. Warm-up

ก่อนเริ่มวัดเวลาจริง ให้โปรแกรมทำงานล่วงหน้าเพื่อให้ Java Virtual Machine (JVM) มีการเตรียมการทำงานก่อนการวัด

3. วัดเวลา

ใช้คำสั่ง

```
long start = System.nanoTime();  
  
// Operation being tested  
  
long end = System.nanoTime();  
  
long time = end - start;
```

เวลาที่ได้คือเวลาที่ใช้ในการทำงานของ Algorithm

4. ทดลอง 5 รอบ

แต่ละขนาดข้อมูลทดลอง 5 รอบ เช่น

Round 1
Round 2
Round 3
Round 4
Round 5

แล้วนำมาหาค่าเฉลี่ย

$$\text{Average} = (\text{Round1} + \text{Round2} + \text{Round3} + \text{Round4} + \text{Round5}) / 5$$

สิ่งที่ต้องวัด

เพื่อให้ตรงกับการวิเคราะห์ Complexity ของงาน สามารถทดลองเปรียบเทียบ Operation หลักได้ดังนี้

Operation	FIFO Queue	Priority Queue
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Display	$O(n)$	$O(n)$

ผลที่คาดว่าจะพบ

เมื่อจำนวน Event เพิ่มขึ้น จะมีแนวโน้มดังนี้

FIFO Queue

ArrayDeque มีการเพิ่มและนำข้อมูลออกจาก Queue ได้รวดเร็ว

Enqueue $\rightarrow O(1)$

Dequeue $\rightarrow O(1)$

Peek $\rightarrow O(1)$

ดังนั้นเมื่อจำนวน Event เพิ่มขึ้น

100 $\rightarrow$ 1,000 $\rightarrow$ 10,000 $\rightarrow$ 50,000

เวลาของ enqueue และ dequeue ไม่ควรเพิ่มขึ้นอย่างเป็นสัดส่วนกับ n

Priority Queue

PriorityQueue ใช้โครงสร้าง Heap ดังนั้น

Enqueue $\rightarrow O(\log n)$

Dequeue $\rightarrow O(\log n)$

Peek $\rightarrow O(1)$

เมื่อจำนวน Event เพิ่มขึ้น การ enqueue และ dequeue จะมีค่าใช้จ่ายมากกว่า FIFO เนื่องจากต้องรักษาลำดับ Priority ของ Heap

สรุป Experimental Comparison

หัวข้อ	FIFO Queue	Priority Queue
หลักการ	เข้าก่อนออกก่อน	Priority สูงออกก่อน
โครงสร้างที่ใช้	ArrayDeque	PriorityQueue
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
เหมาะกับ	งานที่ต้องการความยุติธรรมตามลำดับ	งานที่ต้องจัดการเหตุเร่งด่วน
ข้อดี	เร็วและง่าย	จัดการเหตุสำคัญก่อน
ข้อจำกัด	เหตุ Critical อาจต้องรอ	เหตุ Priority ต่ำอาจรอนาน

จากการทดลองพบว่า FIFO Queue มีประสิทธิภาพสูงในการเพิ่มและนำข้อมูลออก เนื่องจาก enqueue และ dequeue มี Time Complexity เท่ากับ $O(1)$ ในขณะที่ Priority Queue มี Time Complexity เท่ากับ $O(\log n)$ สำหรับ enqueue และ dequeue เนื่องจากต้องรักษาลำดับของ Priority อย่างไรก็ตาม Priority Queue เหมาะสมกับระบบ Emergency Call Dispatch มากกว่าในกรณีที่ต้องให้ความสำคัญกับเหตุฉุกเฉินที่มีความรุนแรงสูง ส่วน FIFO เหมาะกับกรณีที่ต้องการรักษาความยุติธรรมตามลำดับเวลาที่รับแจ้งเหตุ